Program Structures and Algorithms
Project Report

Dwij Patel (002068727) | Sarthak Mallick (002303265)

GITHUB LINK: https://github.com/sarthak-mallick/DSAIPG

INSTRUCTIONS TO RUN:

We use a fork of the existing DSAIPG repository so it's still the same Maven project. Using an IDE like IntelliJ IDEA, or VS Code, you can also run the program after cloning. After Maven downloads all the dependencies, run **MCTS.java** for TicTacToe or **ReversiMCTS.java** for Reversi. Similarly, you can run MCTSTest.java or ReversiMCTSTest.java for test cases. All new files are in Java/src/main/java/com/phasmidsoftware/dsaipg/projects/mcts while the new test cases are in Java/src/test/java/com/phasmidsoftware/dsaipg/projects/mcts

MONTE CARLO TREE SEARCH

Monte Carlo Tree Search (MCTS) is an advanced decision-making technique that plays a crucial role in artificial intelligence, particularly in scenarios that involve uncertainty and vast search spaces. It is extensively applied in strategic games and planning systems where identifying the optimal course of action is challenging. MCTS progressively constructs a decision tree by performing numerous simulations. Through these simulations, it effectively navigates between two competing goals: exploring untested actions to discover new opportunities and exploiting actions already known to yield favorable outcomes. This intelligent balance enables MCTS to refine its decisions over time. The core of the algorithm is built upon four fundamental phases: selection, expansion, simulation, and backpropagation.

1. Selection:

The search begins at the root node, representing the current state of the game. The algorithm traverses the tree by selecting child nodes that maximize a balance between high reward and low visitation, typically using a mathematical formula known as the Upper Confidence Bound applied to Trees (UCT). This approach ensures that both frequently visited and less-explored options are considered.

2. Expansion:

Once the algorithm reaches a node that is not fully expanded and is not a terminal state, it generates one or more child nodes representing the possible next moves. These nodes are added to the search tree for further evaluation.

3. Simulation:

From the newly added node, MCTS conducts a random or semi-random simulation, also known as a rollout. The game is played out to completion, and the result is recorded. These simulations estimate the likely outcome if that particular move were chosen, providing a statistical basis for decision-making

4. Backpropagation:

After each simulation, the outcome (win, loss, or draw) is propagated back up the tree. Nodes along the path to the root are updated with increased visit counts and, where applicable, win counts. This feedback mechanism continuously improves the quality of future move selections.

This iterative loop of selection, expansion, simulation, and backpropagation allows MCTS to focus computational effort on the most promising parts of the search space. Since it can be halted at any point and still provide a reasonably good move, MCTS is considered an anytime algorithm. Its strength lies in being adaptable and statistically guided, improving performance with more computation time.

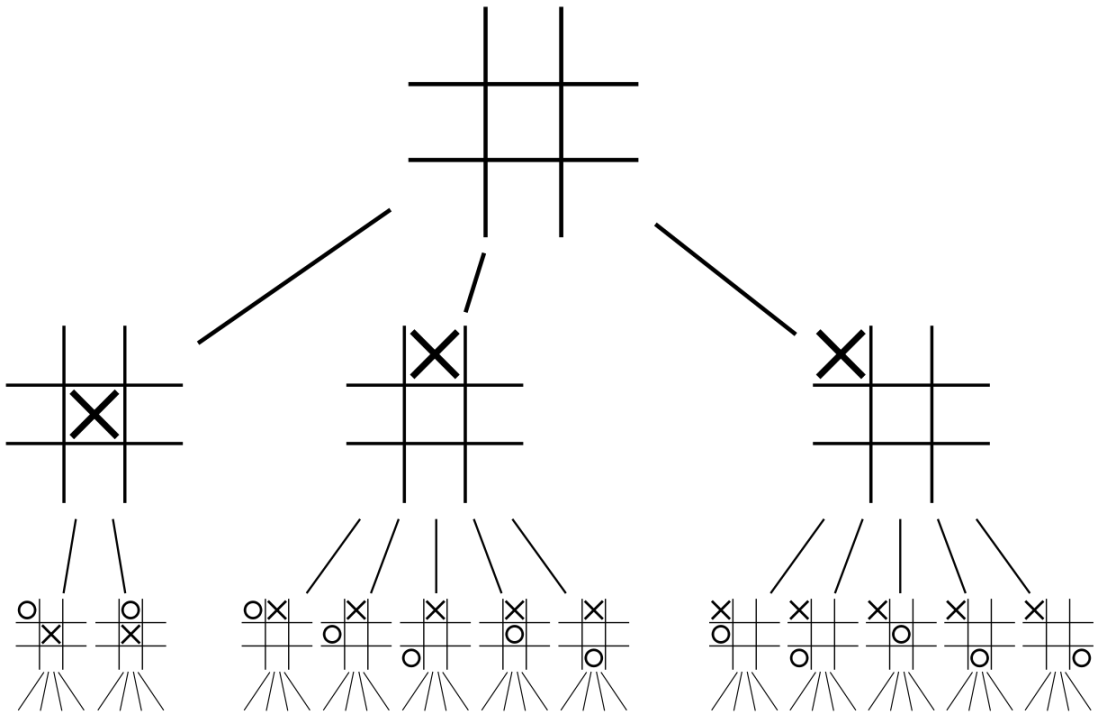
APPLICATION TO TIC-TAC-TOE

Tic-Tac-Toe is a simple yet fully solved game played on a 3x3 grid. Two players take turns placing either 'X' or 'O' in empty cells, aiming to align three of their symbols in a straight line. Due to its limited state space, optimal strategies are well-known, and the game ends in a draw when both players play perfectly.

In this project, we implemented an AI player for Tic-Tac-Toe using MCTS. By simulating thousands of possible game outcomes, the CPU can make intelligent decisions even in such a small game environment. The MCTS-based CPU player evaluates each move not just on immediate gain but on long-term statistical likelihoods of winning.

The AI follows the full MCTS process:
- It starts by selecting a path down the current game tree using UCT.
- It expands new nodes for unexplored legal moves.
- It runs simulations from those new positions to completion.
- It updates node values based on simulation results.

This approach allows the AI to dynamically adapt its strategy as the game progresses, often predicting and blocking winning moves from the opponent or identifying its own winning paths in advance.



IMPLEMENTATION STRATEGY

Each node in the MCTS tree stores the game state, the number of visits, and the number of wins. At each turn, the AI runs a fixed number of simulations (iterations), during which the MCTS process builds and refines the game tree. The best move is selected as the child of the root with the highest visit count.

The structure ensures modularity and clarity, with separate methods for selection, expansion, simulation, and backpropagation. This makes the system scalable and adaptable for more complex games.

SIMULATION POLICY

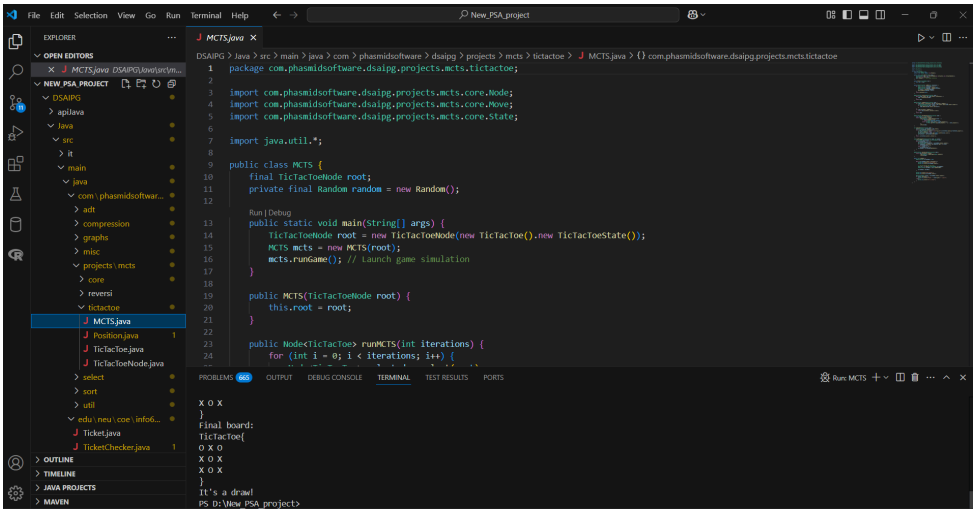
For Tic-Tac-Toe, simulations are conducted randomly due to the small state space. This random policy is effective because it offers a balanced and unbiased estimate of

outcome probabilities from any given node. For larger or more strategic games, the simulation phase can be improved with heuristic or policy-guided simulations.

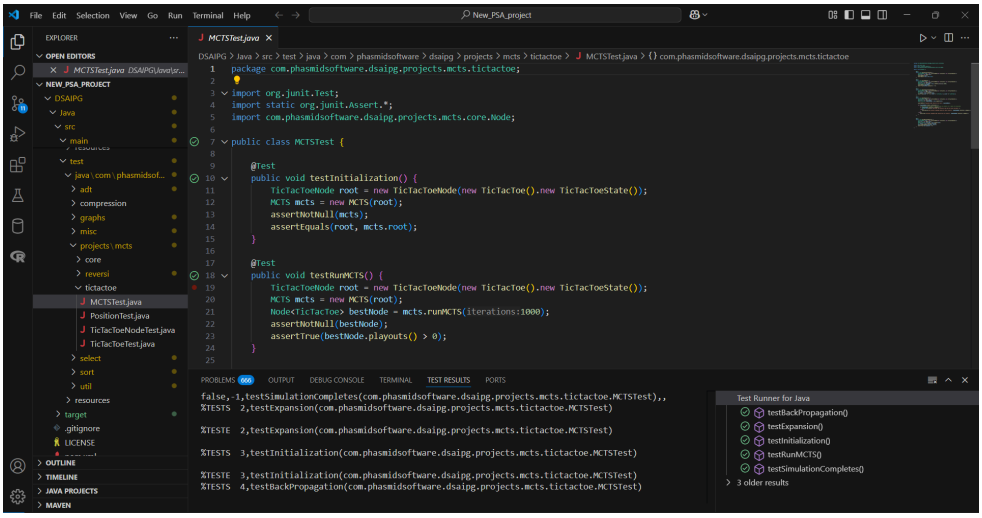
PERFORMANCE AND ACCURACY

Though the simplicity of Tic-Tac-Toe makes brute-force approaches feasible, using MCTS highlights its potential for scalability. The AI can simulate a large number of games quickly and use statistical analysis to make moves that are near-optimal. It reliably avoids poor moves, often forces draws, and occasionally wins against suboptimal play.

OUTPUT SCREENSHOTS:



UNIT TEST SCREENSHOTS:

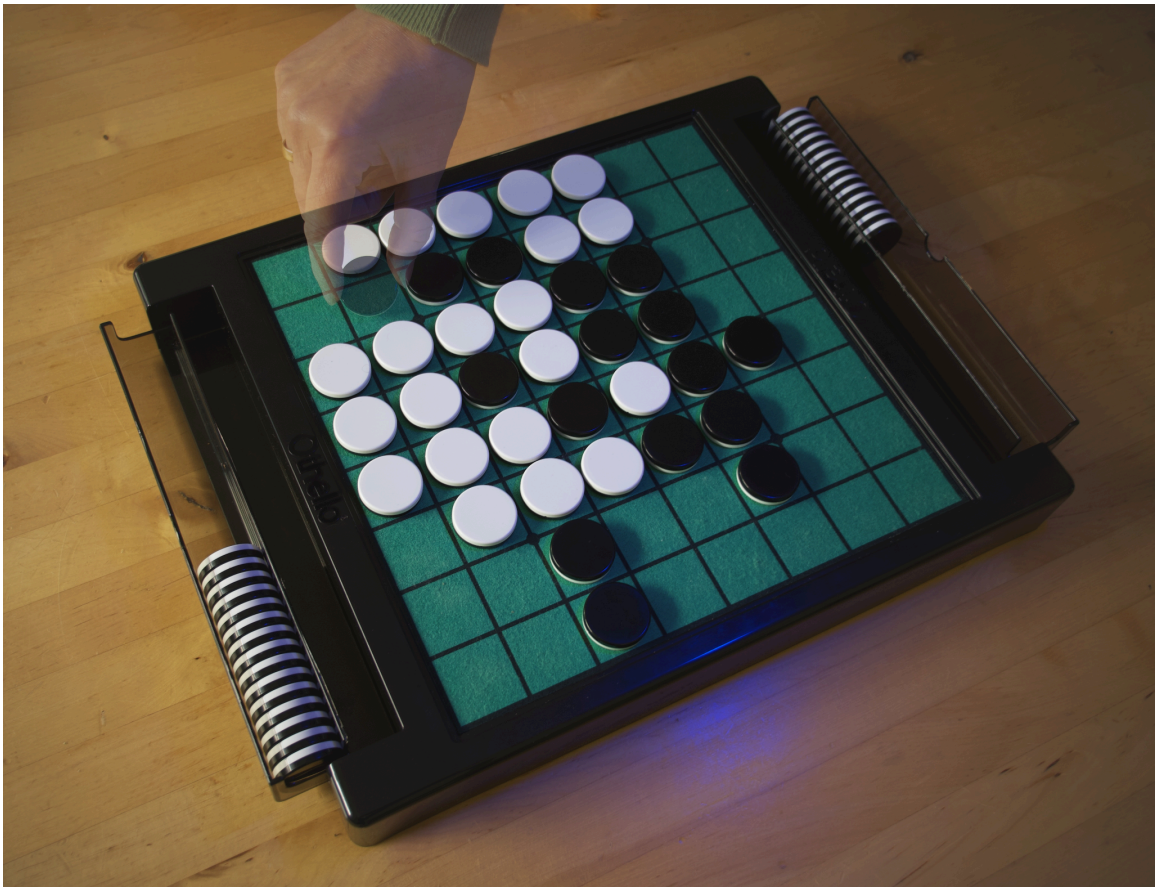


REVERSI GAME EXPLAINED

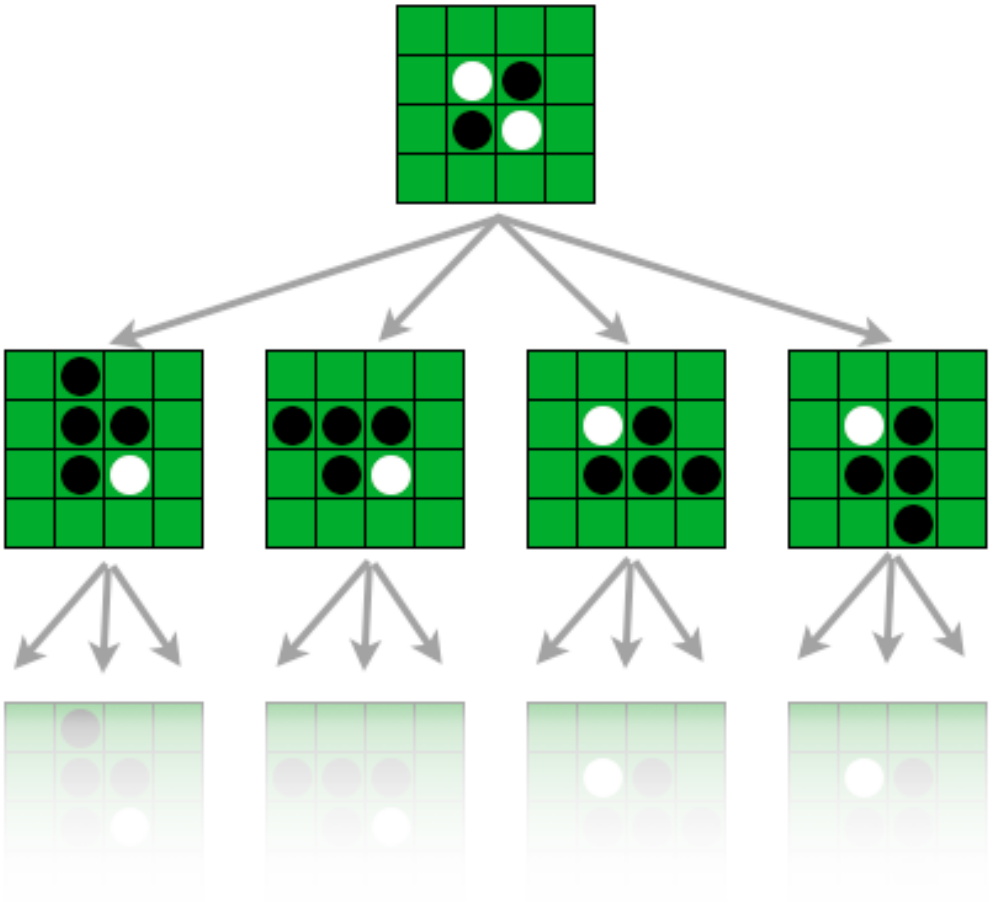
Reversi is a strategy board game for two players, played on an n×n uncheckered board who can place pieces of their own color (Black/White) and flip the opponent's pieces. After a play is made, any opponent’s pieces that lie in a straight line bounded by the current move and another one belonging to the current player are turned over.

Rules:-

- Initial configuration is two pieces each in the center with alternating colors
- Only moves that flip at least one opponent’s piece are valid
- If no valid move exists for the current player turn passes to the opponent
- The game ends when the board is full or no valid moves are left for either player. The player with the most pieces is declared the winner.



REVERSI MCTS IMPLEMENTATION AND HEURISTICS



The high-level implementation structure is similar to that of TicTacToe. Below are some important parameters and method additions to the previous MCS implementation.

Search tree reuse: The existing **runGame()** approach is inefficient for Reversi because:

- We lose all previously learned information about the game tree
- We waste memory and time rebuilding branches we've already explored
- The quality of play suffers because we have less accumulated knowledge

Our approach is reuses the search tree by:

- Instead of creating a new tree for each move, we now retain the entire tree and simply advance to the appropriate subtree after each move
- This preserves all accumulated statistics and explored branches. The algorithm builds on existing knowledge, improving decision quality over time

- Implementation details:
    - Added advanceTree() method to transition to a child node after a move
    - Added findMatchingChild() helper to locate the correct subtree
    - Modified runGame() to reuse the tree between moves

explorationConstant: Parameter that controls the balance between exploration and exploitation in the search algorithm. It's a critical parameter in the UCT formula used during the selection phase of MCTS. In my implementation

$$\text{UCT Score} = \text{wins/child\_playouts} + \sqrt{\text{explorationConstant}} * \sqrt{(\log(\text{parent\_playouts}+1)/\text{child\_playouts})}$$

- A higher value leads to more exploration of less-visited paths in the game tree, potentially discovering new strategies but taking longer to converge
- A lower value exploits known good paths, which can lead to faster convergence but might miss optimal solutions

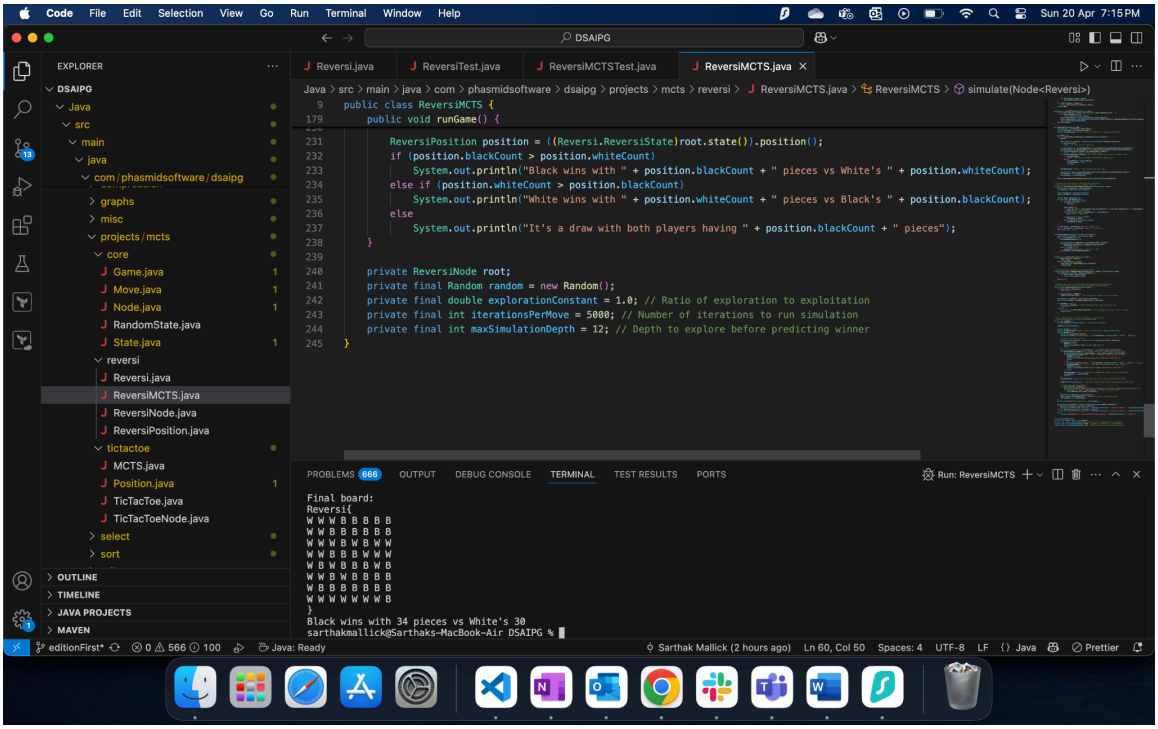
maxSimulationDepth: Parameter that limits how deep the MCTS algorithm explores during the simulation phase. In complex games like Reversi, playing all the way to a terminal state can be computationally expensive. Setting a maximum depth allows:

- Performance Improvement: Random playouts stop after reaching this depth limit, making each simulation faster
- Early Evaluation: Instead of playing to completion, the algorithm evaluates the position at the depth limit using heuristics in evaluatePositionWithHeuristics

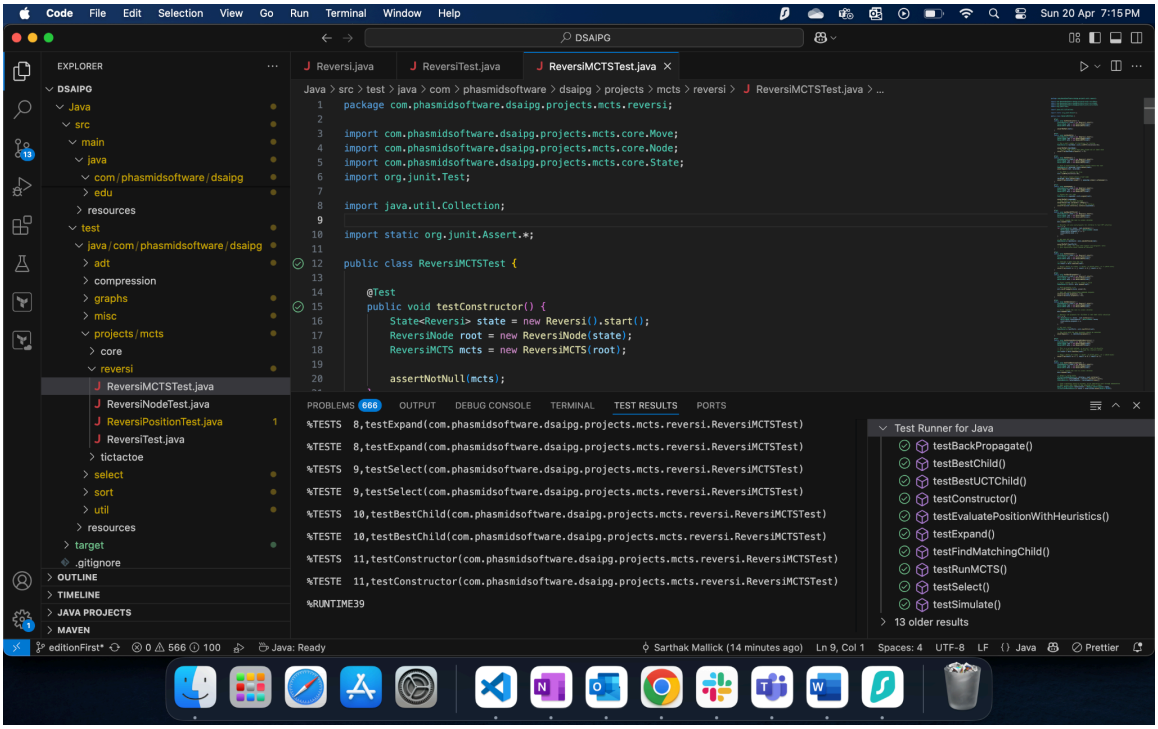
evaluatePositionWithHeuristics: Evaluates a Reversi board position to determine whether Black or White has an advantage during MCTS simulations when we reach a maximum depth. It initializes the scores for black and white equal to the current number of pieces. Then adds extra bonuses if pieces are in strategically valuable positions.

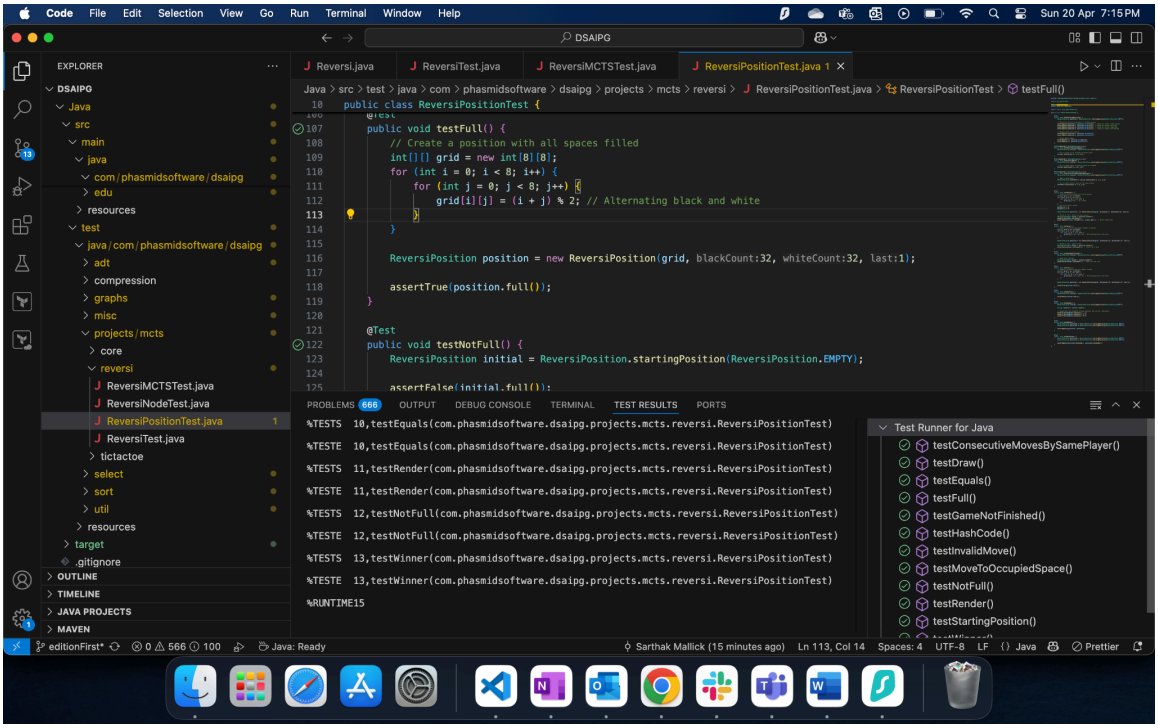
- Corners: Most valuable positions in Reversi because once placed, these pieces can never be flipped. So 2 bonus points per corner occupied a player.
- Edges: Edge pieces are more difficult (though not impossible) to flip than other pieces. So 0.5 bonus points per piece placed on an edge.
- Comparing the scores after adding bonuses, we determine a presumptive winner

OUTPUT SCREENSHOTS:



UNIT TEST SCREENSHOTS:



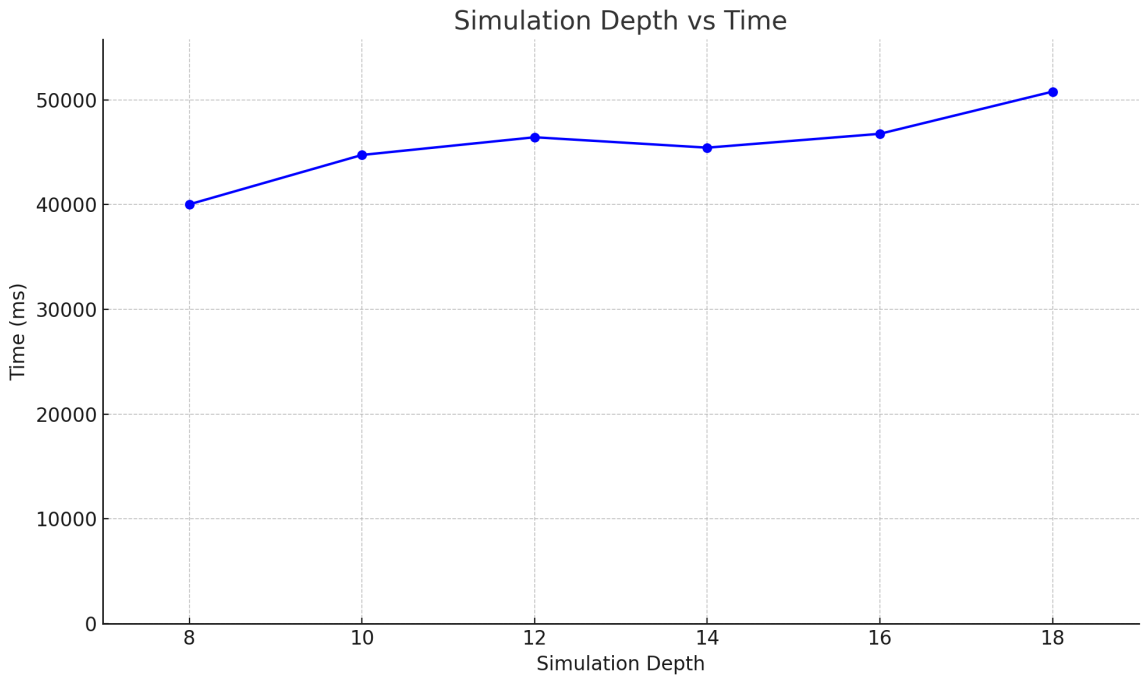


RUN TIME vs SIMULATION DEPTH

Simulation Depth	Time(ms)
8	40018
10	44731
12	46421
14	45428
16	46751
18	50778

All data is for an 8x8 board with the same initial configuration. This benchmark data is plotted in the graph below. The timing is the total time for all moves by both players from start till end.

As we can see, reducing the depth decreases the run time. But it only expands a smaller portion of the possible search tree.



REAL-LIFE APPLICATIONS

MCTS is widely used in games like Go, Chess, and Shogi, which have significantly larger state spaces. It has also found applications in robotics (e.g., motion planning), scheduling problems, autonomous agents, and real-time decision-making systems. Its anytime nature and flexibility make it ideal for environments where computational budgets and time constraints vary.

REFERENCES

1. https://en.wikipedia.org/wiki/Reversi
2. https://royhung.com/reversi
3. https://en.wikipedia.org/wiki/Monte_Carlo_tree_search
4. https://int8.io/monte-carlo-tree-search-beginners-guide/